

Smart Cabin & Vehicle Health Monitor

Technical Documentation

ISO C++17 Multi-Threaded Console Application

Language	C++17
Build Tool	GNU Make + g++
Platform	Windows / Linux / macOS
Threading	std::thread, mutexes, atomics
Version	1.0
Date	May 29, 2026

A real-time vehicle health monitoring system that reads six cabin sensors, evaluates safety conditions on background threads, raises prioritized alerts, logs events asynchronously, and presents an interactive terminal dashboard with driver profiles, statistics, and watchdog health checks.

Repository: `cpp_hack`

Contents

1	Project Overview	5
1.1	Introduction	5
1.2	System Goals	5
1.3	Repository Structure	6
2	Architecture	7
2.1	High-Level Design	7
2.2	MonitorThread Cyclic Evaluation Pipeline	7
2.3	System Architecture Diagram	8
2.4	Background Threads	9
2.5	Startup and Shutdown Sequence	9
2.5.1	Startup	9
2.5.2	Shutdown	9
2.6	Module Dependency Graph	10
2.7	Global Control Flags	10
3	Data Types and Enumerations	12
3.1	Sensor Types	12
3.2	Alert Types and Severities	12
3.3	Driver Profile Structure	12
3.4	Log Levels	13
4	Module Documentation	14
4.1	Sensor Module (<code>sensor.hpp</code> / <code>sensor.cpp</code>)	14
4.1.1	Responsibility	14
4.1.2	Class Hierarchy	14
4.1.3	Input Validation Ranges	15
4.1.4	Critical Display vs. Alert Logic	15
4.2	Alert Module (<code>alert.hpp</code> / <code>alert.cpp</code>)	15
4.2.1	Responsibility	15
4.2.2	Alert Condition Rules	16
4.2.3	Deduplication and Auto-Clear	16
4.2.4	Alert Lifecycle State Machine	16
4.3	Dashboard Module (<code>dashboard.hpp</code> / <code>dashboard.cpp</code>)	17
4.3.1	Responsibility	17
4.3.2	Menu Options	18

4.3.3	Sparklines	18
4.4	Logger Module (<code>logger.hpp</code> / <code>logger.cpp</code>)	18
4.4.1	Responsibility	18
4.4.2	Log Entry Format	18
4.4.3	Priority Queue Ordering	18
4.4.4	Asynchronous Logging Pipeline	19
4.5	Profile Module (<code>profile.hpp</code> / <code>profile.cpp</code>)	19
4.5.1	Responsibility	19
4.5.2	Built-in Profiles	19
4.6	Health Module (<code>health.hpp</code> / <code>health.cpp</code>)	19
4.6.1	Responsibility	20
4.6.2	Watchdog Health State Machine	20
4.7	Main Module (<code>main.cpp</code>)	21
4.7.1	Configuration Loading	21
4.7.2	Auto-Simulation Waveforms	21
5	Concurrency and Thread Safety	22
5.1	Synchronization Primitives	22
5.2	Race Condition Mitigations	22
6	Build, Run, and Test	23
6.1	Requirements	23
6.2	Compilation	23
6.3	Running the Application	23
6.4	Manual Sensor Input Workflow	23
6.5	Automated Test Suite	24
7	Example Scenarios	25
7.1	Normal Driving	25
7.2	Engine Overheat	25
7.3	Door Open While Moving	25
7.4	Watchdog Timeout (Debug)	26
8	Design Patterns and Best Practices	27
8.1	Patterns Used	27
8.2	C++ Features Employed	27
9	Output Files	28
9.1	<code>logs/vehicle_logs.txt</code>	28
9.2	<code>data/profiles.json</code>	28
10	Glossary	29

List of Tables

2.1	MonitorThread Cycle — Step Overview	7
2.2	Background Thread Responsibilities	9
2.3	Global Atomic Flags (<code>main.cpp</code>)	11
3.1	SensorType Enumeration	12
3.2	AlertType Values	12
4.1	Manual Input Validation Bounds	15
4.2	Alert Trigger Conditions (per active profile)	16
4.3	Main Menu (Options 1–8, plus hidden 9)	18
4.4	Default Driver Profiles	19
4.5	Configuration Keys (<code>data/config.txt</code>)	21
4.6	Auto-Simulation Formulas (t = elapsed seconds)	21
5.1	Mutex Usage by Component	22
6.1	Test Module Summary	24
7.1	Safe Sensor Values (Default Profile)	25
8.1	Software Design Patterns	27
10.1	Glossary of Terms	29

List of Figures

2.1	MonitorThread cyclic pipeline — one evaluation cycle	8
2.2	System architecture — threads, shared state, and primary data flows . . .	8
2.3	Module dependency graph — arrows denote header dependency / “uses”. Short labels map to *.hpp/*.cpp pairs; <code>main</code> is <code>main.cpp</code>	10
4.1	Sensor class hierarchy — six concrete types inherit from <code>Sensor</code>	14
4.2	Per-alert-type lifecycle — evaluated every MonitorThread cycle	17
4.3	Asynchronous logging pipeline — producer–consumer pattern	19
4.4	Watchdog state per monitored thread — <code>ECU_FAULT</code> is not auto-cleared when pings resume	20

Chapter 1

Project Overview

1.1 Introduction

The **Smart Cabin & Vehicle Health Monitor** is a C++17 console application that simulates an in-vehicle monitoring unit. It continuously tracks six sensors (engine temperature, battery voltage, vehicle speed, tire pressure, door status, and seatbelt status), evaluates them against configurable safety thresholds, and presents results through a menu-driven terminal dashboard.

The system is designed to mirror real automotive software patterns:

- **Multi-threaded architecture** — sensor simulation, alert evaluation, logging, and health monitoring run concurrently.
- **Thread-safe shared state** — each sensor protects its value with a mutex; managers use their own locks.
- **Asynchronous logging** — producers enqueue log events; a dedicated thread drains the queue to disk.
- **Watchdog supervision** — a health monitor detects unresponsive worker threads.
- **Driver profiles** — different threshold sets (Default, Eco, Sport, Performance) can be switched at runtime.

1.2 System Goals

- Provide a live, color-coded view of all sensor readings and alert status.
- Detect and classify vehicle faults: engine overheat, low/high battery, tire pressure anomalies, overspeed, door open while moving, and seatbelt unbuckled while moving.
- Maintain alert history with deduplication (one active alert per type) and automatic clearing when conditions normalize.
- Record min/max/average statistics and ASCII sparkline trends for numeric sensors.
- Support manual sensor input for testing and optional sinusoidal auto-simulation.

- Persist structured event logs to `logs/vehicle_logs.txt` with rotation at 1 MB.
- Allow graceful shutdown via menu or SIGINT (Ctrl+C).

1.3 Repository Structure

```

cpp_hack/
  include/
    sensor.hpp      # Public header files
    alert.hpp       # Sensor hierarchy and SensorType enum
    dashboard.hpp   # Alert, AlertManager, severity/type enums
    logger.hpp      # Dashboard UI and VehicleStatistics
    profile.hpp     # EventLogger, LogEntry, LOG_* macros
    health.hpp      # DriverProfile and ProfileManager
  src/              # HealthMonitor (watchdog)
    main.cpp        # Implementation files
    sensor.cpp
    alert.cpp
    dashboard.cpp
    logger.cpp
    profile.cpp
    health.cpp
  data/
    config.txt      # Entry point, threads, menu loop
    profiles.json   # Timing and threshold defaults (partial use)
  logs/
    vehicle_logs.txt # Driver profile definitions
  tests/
    test_cases.py   # Runtime event log (created on first run)
    test_runner.py  # 45 automated test definitions
    test_report.md  # Parallel subprocess test harness
  Makefile          # Generated Markdown report
  vehicle_monitor   # Compiled executable (or .exe on Windows)
  documentation.tex # This document

```

Chapter 2

Architecture

2.1 High-Level Design

The application splits work across **one main thread** (interactive menu and dashboard rendering) and **four background worker threads** managed by a `ThreadManager` RAII helper that joins all threads on destruction. Unlike a single-threaded cyclic ECU, this system uses *parallel workers* that share sensor and manager objects protected by mutexes.

2.2 MonitorThread Cyclic Evaluation Pipeline

The closest analogue to a cyclic ECU pipeline is the `MonitorThread` loop in `main.cpp`. Every `ALERT_CHECK_INTERVAL` milliseconds (default 750 ms), it runs `alertMgr.evaluateConditions()` when sensors have been initialized:

Table 2.1: MonitorThread Cycle — Step Overview

Step	Action	Description
1	<code>ping("MonitorThread")</code>	Records heartbeat for the software watchdog.
2	Sensor-init gate	If <code>g_sensorsInitialized</code> is false, sleep and skip evaluation (avoids alerts on zero defaults).
3	Read sensor values	Iterates all six sensors; copies current readings into local variables.
4	Load profile thresholds	Reads <code>currentProfile_</code> (engine, battery, tire, speed, door-speed limits).
5	<code>checkCondition() × 8</code>	Evaluates each alert rule; raises, deduplicates, or clears alerts; emits <code>LOG_*</code> .
6	Trim history	Removes oldest entries if <code>alertHistory_</code> exceeds <code>maxHistory_</code> .
7	Sleep	<code>sleep_for(alertInterval)</code> ; loop repeats while <code>g_running</code> .

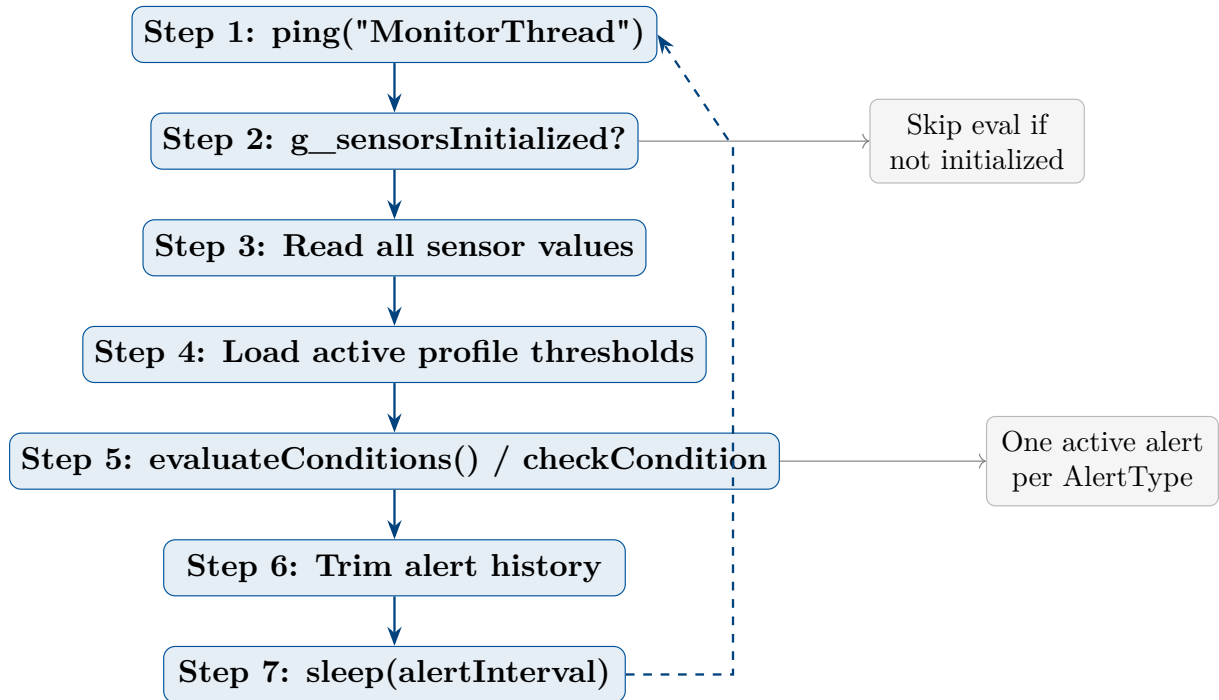


Figure 2.1: MonitorThread cyclic pipeline — one evaluation cycle

2.3 System Architecture Diagram

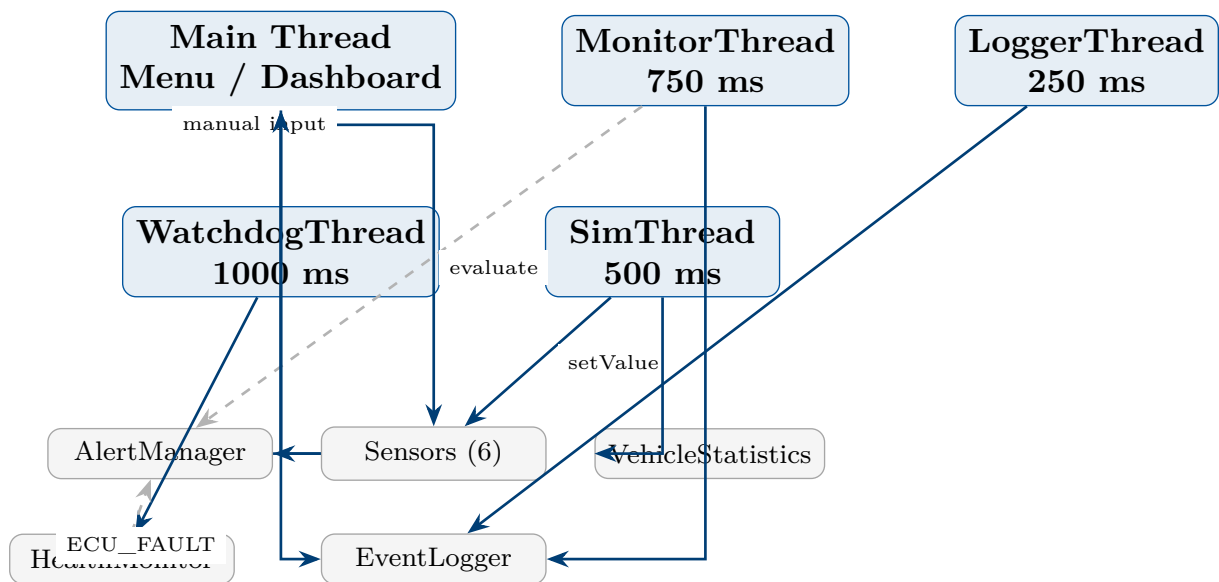


Figure 2.2: System architecture — threads, shared state, and primary data flows

2.4 Background Threads

Table 2.2: Background Thread Responsibilities

Thread	Interval	Behaviour
MonitorThread	750 ms (configurable)	Calls <code>alertMgr.evaluateConditions(sensors)</code> when <code>g_sensorsInitialized</code> is true. Skips evaluation until sensors have been set manually or by simulation.
LoggerThread	250 ms	Drains <code>EventLogger</code> 's priority queue and appends entries to <code>logs/vehicle_logs.txt</code> . Supports debug hang simulation via <code>g_simulateHang</code> .
WatchdogThread	1000 ms	Calls <code>healthMonitor.checkHealth(alertMgr)</code> . Raises <code>ECU_FAULT</code> if any component misses heartbeat for > 2000 ms.
SimThread	500 ms	When <code>g_autoSimulate</code> is ON, updates numeric sensors with sinusoidal waveforms and randomly toggles door/seatbelt (5% chance per cycle).

2.5 Startup and Shutdown Sequence

2.5.1 Startup

1. Register SIGINT handler (`signalHandler`) to set `g_running = false`.
2. Create `EventLogger` pointing to `logs/vehicle_logs.txt`; register as global logger.
3. Load `data/config.txt` (or use defaults on failure).
4. Instantiate six `unique_ptr<Sensor>` objects.
5. Load driver profiles from `data/profiles.json`; apply active profile thresholds to `AlertManager`.
6. Seed `VehicleStatistics` with initial sensor values.
7. Start four background threads; wait 2 seconds; enter main menu loop.

2.5.2 Shutdown

1. User selects menu option 7, presses Ctrl+C, or stdin EOF.
2. Set `g_running = false`; notify logger queue.

3. `ThreadManager` destructor joins all threads.
4. Flush pending log events; print shutdown summary (sensor count, alert count, event count).
5. Clear global logger pointer.

2.6 Module Dependency Graph

The module layout forms a directed acyclic graph. `profile.hpp` defines shared threshold types; `logger.hpp` has no upstream module dependencies.

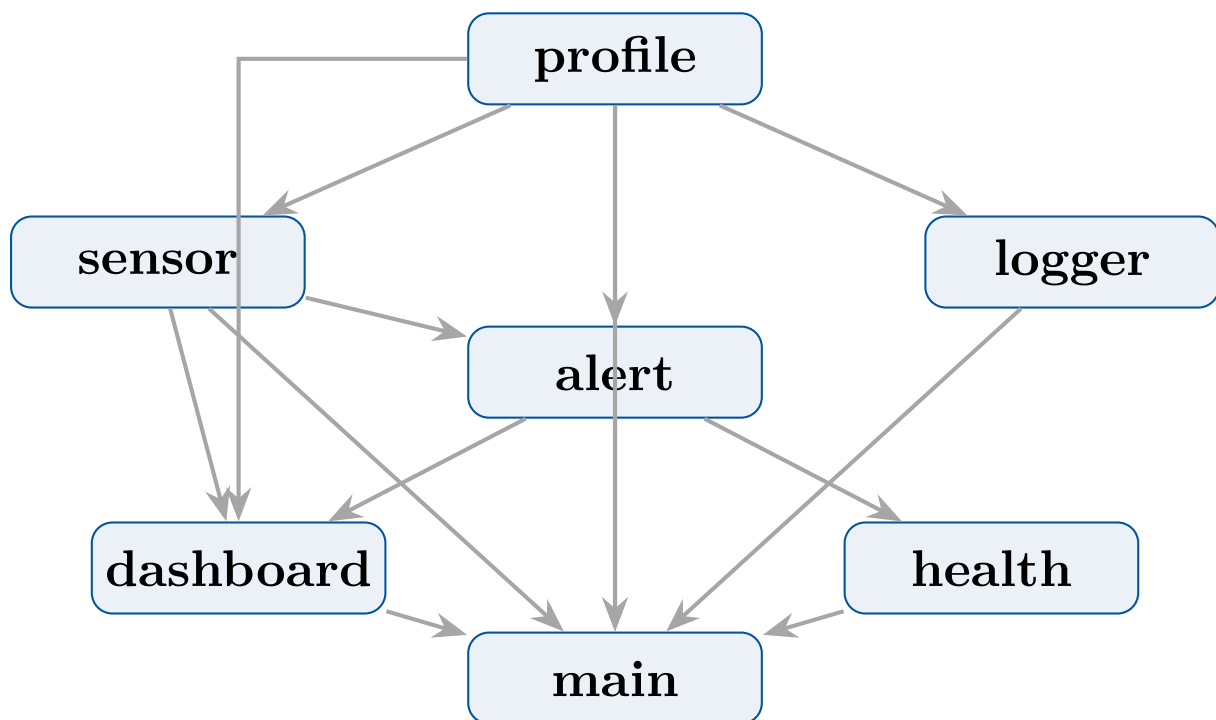


Figure 2.3: Module dependency graph — arrows denote header dependency / “uses”. Short labels map to `*.hpp/*.cpp` pairs; `main` is `main.cpp`.

2.7 Global Control Flags

Four `std::atomic<bool>` globals coordinate cross-thread behaviour:

Table 2.3: Global Atomic Flags (`main.cpp`)

Flag	Purpose
<code>g_running</code>	Master shutdown signal for all threads and menu loop.
<code>g_sensorsInitialized</code>	Set after manual input or auto-sim; gates alert evaluation.
<code>g_autoSimulate</code>	Enables/disables sinusoidal sensor updates in <code>SimThread</code> .
<code>g_simulateHang</code>	Debug: <code>LoggerThread</code> sleeps 10 s to trigger watchdog.

Chapter 3

Data Types and Enumerations

3.1 Sensor Types

Table 3.1: SensorType Enumeration

Enumerator	Sensor Class	Unit / Encoding
EngineTemp	EngineTemperatureSensor	°C (default 0)
BatteryVoltage	BatterySensor	Volts
VehicleSpeed	SpeedSensor	km/h
TirePressure	TirePressureSensor	PSI
DoorStatus	DoorSensor	0 = CLOSED, 1 = OPEN
Seatbelt	SeatbeltSensor	0 = LOCKED, 1 = UNLOCKED

3.2 Alert Types and Severities

Table 3.2: AlertType Values

Type	Default Severity
ENGINE_OVERHEAT	CRITICAL
LOW_BATTERY	WARNING
HIGH_BATTERY	WARNING
LOW_TIRE_PRESSURE	WARNING
HIGH_TIRE_PRESSURE	WARNING
OVERSPEED	WARNING
DOOR_OPEN	CRITICAL
SEATBELT_WARNING	WARNING
ECU_FAULT	CRITICAL (watchdog only)

3.3 Driver Profile Structure

```
1 struct DriverProfile {  
2     std::string name;  
3     double engineThreshold;    // C      alert if temp > this  
4     double batteryThreshold;   // V      alert if voltage < this  
5     double tireThreshold;      // PSI    alert if pressure < this  
6     double speedLimit;         // km/h   alert if speed > this  
7     double doorSpeedThreshold; // km/h   door-open alert if speed >  
        this  
8 };
```

Listing 3.1: DriverProfile struct (profile.hpp)

3.4 Log Levels

LogLevel: DEBUG, INFO, WARNING, CRITICAL. Mapped to 8-character strings in log output. Higher severity entries are dequeued first from the priority queue.

Chapter 4

Module Documentation

4.1 Sensor Module (`sensor.hpp` / `sensor.cpp`)

4.1.1 Responsibility

The sensor module defines an abstract `Sensor` base class and six concrete implementations. Each sensor:

- Stores name, type, unit, and current value behind a mutable `std::mutex`.
- Exposes the public API: `getValue()`, `setValue()`, `display()`, `getFormattedValue()`, and `isCritical(profile)`.
- Validates manual input ranges via `validateInput()`.

A static `std::atomic<int> totalSensorCount_` tracks live sensor instances (incremented in constructor, decremented in destructor).

4.1.2 Class Hierarchy

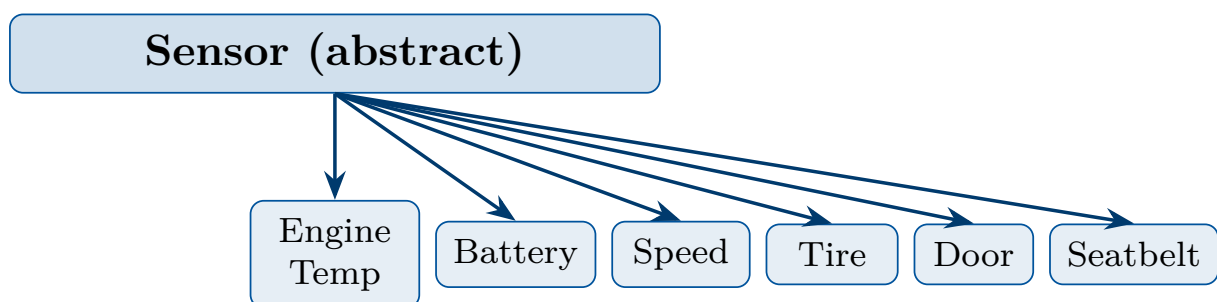


Figure 4.1: Sensor class hierarchy — six concrete types inherit from `Sensor`

Each derived class overrides `display()`, `getFormattedValue()`, and `isCritical()`; it also implements `validateInput()` where validation differs from the base class.

4.1.3 Input Validation Ranges

Table 4.1: Manual Input Validation Bounds

Sensor	Valid Range	Notes
Engine Temperature	−50 to 200 °C	Hard reject outside range
Battery Voltage	0 to 20 V	Warn if < 13.5 V or > 15 V
Vehicle Speed	0 to 300 km/h	
Tire Pressure	0 to 50 PSI	Warn if > 40 PSI
Door Status	exactly 0 or 1	
Seatbelt Status	exactly 0 or 1	

4.1.4 Critical Display vs. Alert Logic

Important distinction: The `operator«` stream formatter on `Sensor` uses *hard-coded* thresholds (e.g. engine > 110°C) for console coloring. Actual alerts use thresholds from the **active driver profile** via `AlertManager`. These can differ when a non-Default profile is selected.

4.2 Alert Module (`alert.hpp` / `alert.cpp`)

4.2.1 Responsibility

`AlertManager` is the core safety engine. Every `alertInterval` milliseconds (default 750 ms), `evaluateConditions()`:

1. Reads all six sensor values into local variables.
2. Compares each against `currentProfile_` thresholds.
3. Uses a lambda `checkCondition` to add or remove alerts with deduplication.
4. Logs trigger/clear events via `LOG_CRITICAL` / `LOG_WARNING` / `LOG_INFO`.
5. Trims `alertHistory_` to `maxHistory_` entries.

4.2.2 Alert Condition Rules

Table 4.2: Alert Trigger Conditions (per active profile)

Alert	Trigger Condition	Severity
Engine Overheat	<code>engineTemp > profile.engineThreshold</code>	CRITICAL
Low Battery	<code>batteryVolts < profile.batteryThreshold</code>	WARNING
High Battery	<code>batteryVolts > 15.0 V</code>	WARNING
Low Tire Pressure	<code>tirePressure < profile.tireThreshold</code>	WARNING
High Tire Pressure	<code>tirePressure > 45.0 PSI</code>	WARNING
Overspeed	<code>speed > profile.speedLimit</code>	WARNING
Door Open	<code>door == 1 AND speed > profile.doorSpeedThreshold</code>	CRITICAL
Seatbelt Warning	<code>seatbelt == 1 AND speed > 0</code>	WARNING
ECU Fault	Watchdog: thread heartbeat > 2000 ms	CRITICAL

4.2.3 Deduplication and Auto-Clear

Deduplication: `isAlertActive(type)` prevents duplicate entries of the same `AlertType` in `activeAlerts_`.

Auto-clear: When a condition becomes false, matching alerts are erased from `activeAlerts_` and an INFO log records the clearance. History retains all past alert objects.

4.2.4 Alert Lifecycle State Machine

Each `AlertType` follows the same per-cycle lifecycle inside `checkCondition()`:

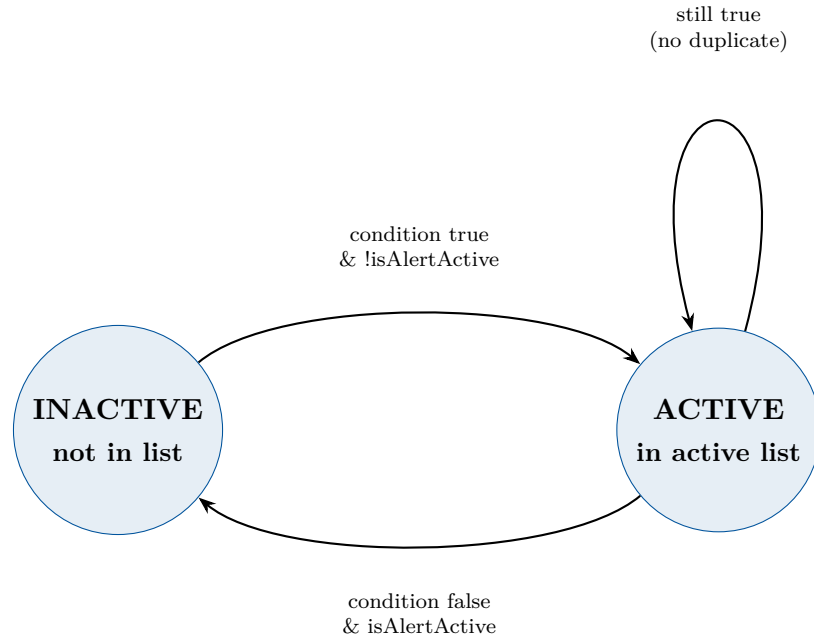


Figure 4.2: Per-alert-type lifecycle — evaluated every MonitorThread cycle

Raised alerts are also appended to `alertHistory_` (never removed from history when cleared from the active list).

```

1 auto checkCondition = [&](bool condition, AlertType type,
2   AlertSeverity severity, const std::string& msg, const std::string&
3   logName) {
4   if (condition) {
5       if (!isAlertActive(type)) {
6           auto alert = std::make_shared<Alert>(type, severity, msg);
7           activeAlerts_.push_back(alert);
8           alertHistory_.push_back(alert);
9           // LOG_CRITICAL or LOG_WARNING ...
10      }
11  } else {
12      if (isAlertActive(type)) { /* LOG_INFO cleared */ }
13      activeAlerts_.erase(/* remove matching type */);
14  };

```

Listing 4.1: Core checkCondition pattern from alert.cpp

4.3 Dashboard Module (`dashboard.hpp` / `dashboard.cpp`)

4.3.1 Responsibility

The dashboard module provides all user-visible terminal UI:

- **VehicleStatistics** — tracks per-sensor min, max, running average, and last 20 readings for sparklines.
- **Dashboard** — renders bordered ASCII tables with ANSI colors (green/yellow/red).

4.3.2 Menu Options

Table 4.3: Main Menu (Options 1–8, plus hidden 9)

Key	Action
1	Live dashboard (refresh with R, toggle sim with Q)
2	View active alerts
3	Alert history (last 20)
4	Statistics + sparkline trends
5	Search event log (by severity or keyword)
6	Manual sensor input
7	Exit (graceful shutdown)
8	Manage driver profiles
9	<i>Debug:</i> simulate LoggerThread hang (10 s)

4.3.3 Sparklines

Numeric sensors display Unicode block characters (\u2581–\u2588) normalized to the min–max range of the last 20 samples. Door and seatbelt are excluded from statistics tracking.

4.4 Logger Module (`logger.hpp` / `logger.cpp`)

4.4.1 Responsibility

`EventLogger` implements producer–consumer logging:

- Producers call `LOG_INFO`, `LOG_WARNING`, etc., which enqueue `LogEntry` objects.
- `LoggerThread` calls `processPendingEvents()` to write to disk and append to in-memory `eventHistory_`.
- Timestamps use IST offset (+5:30) applied before formatting.
- Log rotation renames the file when size exceeds 1 MB.

4.4.2 Log Entry Format

```
[YYYY-MM-DD HH:MM:SS] [LEVEL    ] [Source | file.cpp:function] Message
```

4.4.3 Priority Queue Ordering

`LogEntry::operator<` makes `std::priority_queue` pop CRITICAL before WARNING before INFO, and earlier timestamps before later ones at the same level.

4.4.4 Asynchronous Logging Pipeline

Logging is decoupled from producers: any thread calling `LOG_*` enqueues work; `LoggerThread` drains the queue to disk.

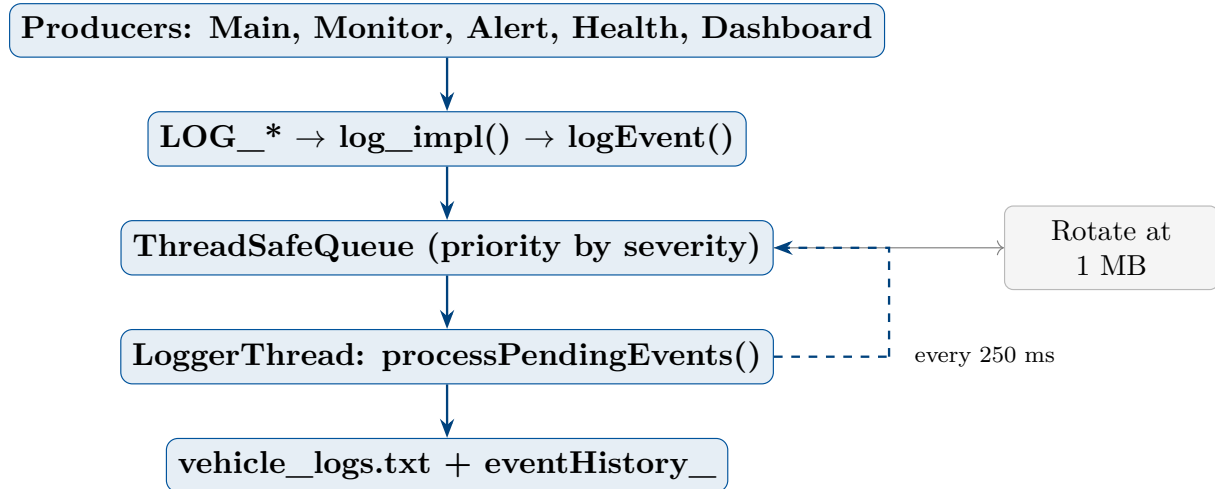


Figure 4.3: Asynchronous logging pipeline — producer–consumer pattern

4.5 Profile Module (`profile.hpp` / `profile.cpp`)

4.5.1 Responsibility

`ProfileManager` loads and saves driver profiles from `data/profiles.json` using a lightweight manual JSON parser (no external library). If the file is missing, four default profiles are created and written to disk.

4.5.2 Built-in Profiles

Table 4.4: Default Driver Profiles

Name	Engine°C	Batt. V	Tire PSI	Speed	Door@km/h
Default	110	11.0	25	120	10
Eco	100	11.5	30	90	0
Sport	125	10.5	22	150	10
Performance	135	10.0	20	180	15

Switching profiles (menu 8) calls `alertMgr.updateThresholds()` immediately; active alerts re-evaluate on the next monitor cycle.

4.6 Health Module (`health.hpp` / `health.cpp`)

4.6.1 Responsibility

`HealthMonitor` implements a software watchdog:

- Each worker thread calls `ping("ThreadName")` every loop iteration.
- `checkHealth()` compares elapsed time since last ping against a 2000 ms timeout.
- On timeout, raises a single `ECU_FAULT CRITICAL` alert (no spam if already active).

Monitored components: `MonitorThread`, `LoggerThread`, `SimThread`.

4.6.2 Watchdog Health State Machine

For each registered component (e.g. `MonitorThread`, `LoggerThread`, `SimThread`):

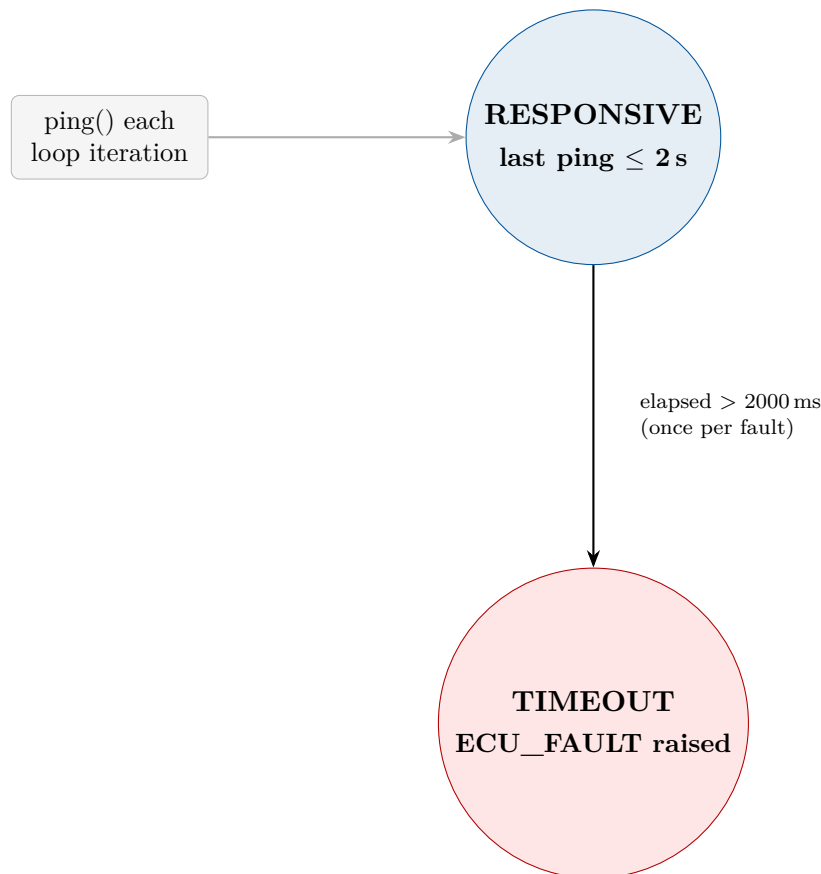


Figure 4.4: Watchdog state per monitored thread — `ECU_FAULT` is not auto-cleared when pings resume

Note: `ECU_FAULT` is raised by `HealthMonitor` but is not cleared automatically when pings resume. It remains in `activeAlerts_` until removed by other logic or process restart.

4.7 Main Module (`main.cpp`)

4.7.1 Configuration Loading

`loadConfig()` parses `data/config.txt` as KEY=VALUE lines (comments with #, whitespace trimmed).

Table 4.5: Configuration Keys (`data/config.txt`)

Key	Default	Used By
ALERT_CHECK_INTERVAL	750 ms	MonitorThread sleep
MAX_ALERT_HISTORY	100	AlertManager deque size
ENGINE_TEMP_CRITICAL	110	<i>Loaded but unused*</i>
BATTERY_VOLTAGE_LOW	11	<i>Loaded but unused*</i>
TIRE_PRESSURE_LOW	25	<i>Loaded but unused*</i>
SPEED_LIMIT	120	<i>Loaded but unused*</i>
DOOR_SPEED_THRESHOLD	10	<i>Loaded but unused*</i>
DASHBOARD_REFRESH_INTERVAL	1000	<i>Not referenced in code</i>

Configuration note: Threshold keys in `config.txt` are parsed in `main()` but alert evaluation uses **driver profile** thresholds from `profiles.json` exclusively. To change alert limits at runtime, switch profiles (menu 8) or edit `profiles.json`.

4.7.2 Auto-Simulation Waveforms

When enabled, `SimThread` applies:

Table 4.6: Auto-Simulation Formulas (t = elapsed seconds)

Sensor	Formula
Engine Temp	$85 + 30 \sin(0.2t)$ °C
Vehicle Speed	$\max(0, 60 + 60 \sin(0.5t))$ km/h
Battery	$12.5 + 2 \sin(0.2t)$ V
Tire Pressure	$32 + 5 \sin(0.2t)$ PSI
Door/Seatbelt	5% random toggle per 500 ms cycle

Chapter 5

Concurrency and Thread Safety

5.1 Synchronization Primitives

Table 5.1: Mutex Usage by Component

Component	Protected Data
Each Sensor	<code>currentValue_</code> , <code>name</code> , <code>unit</code>
AlertManager	<code>activeAlerts_</code> , <code>alertHistory_</code> , <code>currentProfile_</code>
VehicleStatistics	All maps and dequeues
Dashboard	Render methods (prevents interleaved output)
EventLogger	<code>eventHistory_</code> , queue internals
HealthMonitor	<code>heartbeats_</code> map

5.2 Race Condition Mitigations

- Manual input disables auto-simulation to prevent `SimThread` overwriting user values within 500 ms.
- Alert evaluation waits for `g_sensorsInitialized` so zero-default sensors do not spuriously alert at startup.
- Atomic `g_running` provides a memory-safe shutdown flag visible to all threads.

Chapter 6

Build, Run, and Test

6.1 Requirements

- g++ with C++17 support
- GNU Make
- POSIX threads (-pthread flag)
- Python 3 (for automated tests)

6.2 Compilation

```
1 make                # produces vehicle_monitor (or vehicle_monitor.exe)
2 make clean          # removes build/ and executable
```

Listing 6.1: Build commands

Compiler flags: -std=c++17 -Wall -Wextra -Wpedantic -O2 -pthread.

6.3 Running the Application

```
1 ./vehicle_monitor    # Linux/macOS
2 vehicle_monitor.exe  # Windows
```

Listing 6.2: Run from project root

Ensure `data/config.txt` and `data/profiles.json` exist relative to the working directory. The `logs/` directory is created implicitly when the logger opens the file.

6.4 Manual Sensor Input Workflow

From the main menu, select option **6**. Enter values in this order:

1. Engine Temperature (°C)

2. Battery Voltage (V)
3. Vehicle Speed (km/h)
4. Tire Pressure (PSI)
5. Door Status (0 or 1)
6. Seatbelt Status (0 or 1)

Press Enter on any line to skip (keep current value). Type **S** to toggle auto-simulation.

6.5 Automated Test Suite

The `tests/` directory contains 45 test cases across six modules:

Table 6.1: Test Module Summary

Prefix	Module	Count
TC-SEN	Sensor threshold boundaries	20
TC-ALT	Alert lifecycle and priority	7
TC-VAL	Input validation	8
TC-CFG	Configuration file parsing	4
TC-LOG	Logger file behaviour	4
TC-DSH	Dashboard display	2

Run tests:

```

1 cd tests
2 python test_runner.py

```

This compiles the project, runs cases in parallel (8 workers, 15s timeout each), and writes `tests/test_report.md`.

Chapter 7

Example Scenarios

7.1 Normal Driving

Table 7.1: Safe Sensor Values (Default Profile)

Sensor	Value
Engine Temperature	85 °C
Battery Voltage	14.0 V
Vehicle Speed	72 km/h
Tire Pressure	32 PSI
Door	CLOSED (0)
Seatbelt	LOCKED (0)

Result: No active alerts; dashboard shows green [NORMAL] status.

7.2 Engine Overheat

- Input: Engine = 115 °C (Default profile threshold = 110 °C)
- Alert: ENGINE OVERHEAT (CRITICAL)
- Log: LOG_CRITICAL with message *“Temperature 115.0 C exceeds 110.0 C”*
- Dashboard: Engine row shows [CRITICAL] in red

7.3 Door Open While Moving

- Door = OPEN (1), Speed = 50 km/h, Default doorSpeedThreshold = 10
- Alert: DOOR OPEN (CRITICAL) — message includes speed and limit
- Door OPEN at 0 km/h: **no alert** (speed must exceed threshold)

7.4 Watchdog Timeout (Debug)

- Menu option 9 sets `g_simulateHang = true`
- `LoggerThread` sleeps 10 seconds without pinging
- Watchdog detects > 2000 ms silence
- Alert: ECU FAULT (CRITICAL) — *“LoggerThread is unresponsive...”*

Chapter 8

Design Patterns and Best Practices

8.1 Patterns Used

Table 8.1: Software Design Patterns

Pattern	Application
Polymorphism	<code>Sensor</code> base class with virtual <code>display()</code> , <code>isCritical()</code>
RAII	<code>ThreadManager</code> joins threads in destructor
Producer–Consumer	<code>LOG_*</code> macros enqueue; <code>LoggerThread</code> consumes
Strategy	Driver profiles swap threshold sets without code changes
Singleton (lightweight)	Global <code>EventLogger*</code> via <code>setGlobalLogger()</code>
Template Method	Alert evaluation lambda reused for all condition types

8.2 C++ Features Employed

- `std::unique_ptr` for sensor ownership in `main`
- `std::shared_ptr<Alert>` for shared alert objects in history and active lists
- `std::atomic` for lock-free flags
- `std::thread` and `std::chrono` for timing
- Move semantics on `Alert` and `LogEntry`
- Deleted copy on `EventLogger` (non-copyable logger)

Chapter 9

Output Files

9.1 logs/vehicle_logs.txt

Append-only event log. Each session begins with a banner:

```
=====
Vehicle Health Monitor - Log Session Started
Timestamp: YYYY-MM-DD HH:MM:SS
=====
```

Individual events follow the standard format described in Chapter 5 (Logger Module). On shutdown, a footer records total events. Files exceeding 1 MB are renamed with an embedded timestamp and a fresh log is started.

9.2 data/profiles.json

JSON array of profile objects. Edited manually or regenerated when defaults are created. Runtime profile selection is not persisted across restarts (always starts at index 0 = Default).

Chapter 10

Glossary

Term	Definition
Alert	A timestamped record of a detected fault condition with type, severity, and message.
Active Alert	An alert currently present in <code>AlertManager::activeAlerts_</code> .
Driver Profile	A named set of threshold values tuning alert sensitivity.
ECU	Electronic Control Unit; here, simulated fault when a software thread hangs.
Heartbeat	A periodic <code>ping()</code> from a worker thread to the health monitor.
Sparkline	A compact ASCII graph of the last 20 sensor readings.
Watchdog	Software that resets or alerts when a component stops responding.
IST	Indian Standard Time; logger applies +5:30 offset to timestamps.
ANSI Escape	Terminal color codes (e.g. <code>\033[32m</code> for green).
Auto-Simulation	Background sinusoidal sensor value generation for demo/testing.
Last-Valid Hold	Pressing Enter during manual input skips a sensor, keeping its current value.

Table 10.1: Glossary of Terms